StruggleForum: Minimal Viable Product Specification

March 1, 2026

Contents

1 Introduction **3**
1.1 Technology stack . . . 3
1.2 Backend structure . . . 3

2 Data Models **3**
2.1 User . . . 4
2.2 Session . . . 5
2.3 Post . . . 6
2.4 Comment . . . 7
2.5 Category . . . 9
2.6 Notification . . . 9
2.7 Like . . . 11

3 API Endpoints **12**
3.1 Authentication Endpoints . . . 12
3.1.1 POST /api/auth/register . . . 12
3.1.2 POST /api/auth/login . . . 13
3.1.3 POST /api/auth/logout . . . 13
3.1.4 GET /api/auth/me . . . 14
3.2 Account Endpoints . . . 14
3.2.1 GET /api/account/profile . . . 14
3.2.2 PATCH /api/account/profile . . . 14
3.2.3 PATCH /api/account/password . . . 15
3.3 User Endpoints . . . 15
3.3.1 GET /api/users/:id . . . 15
3.3.2 GET /api/users/:id?include=posts . . . 15
3.4 Post Endpoints . . . 16
3.4.1 GET /api/posts . . . 16
3.4.2 GET /api/posts/:id . . . 16
3.4.3 POST /api/posts . . . 16
3.4.4 PUT /api/posts/:id . . . 17
3.4.5 DELETE /api/posts/:id . . . 18
3.4.6 POST /api/posts/:id/lock . . . 18
3.4.7 POST /api/posts/:id/unlock . . . 18
3.5 Comment Endpoints . . . 18
3.5.1 GET /api/posts/:id/comments . . . 18
3.5.2 POST /api/posts/:id/comments . . . 19
3.5.3 PUT /api/comments/:id . . . 19

3.5.4 DELETE /api/comments/:id . . . 20
3.5.5 POST /api/comments/:id/lock . . . 20
3.5.6 POST /api/comments/:id/unlock . . . 20
3.6 Like Endpoints . . . 20
3.6.1 POST /api/likes . . . 20
3.6.2 DELETE /api/likes/:id . . . 21
3.7 Notification Endpoints . . . 21
3.7.1 GET /api/notifications . . . 21
3.7.2 PUT /api/notifications/:id/read . . . 21
3.8 Category Endpoints . . . 22
3.8.1 GET /api/categories . . . 22
3.8.2 POST /api/categories . . . 22
3.9 Admin Endpoints . . . 22
3.9.1 GET /api/admin/users . . . 22
3.9.2 PATCH /api/admin/users/:id/role . . . 23
3.9.3 DELETE /api/admin/posts/:id . . . 23
3.9.4 DELETE /api/admin/comments/:id . . . 23

4 Permission Rules **24**
4.1 Responsibility Split . . . 24
4.2 User Permissions . . . 24
4.3 Admin Permissions . . . 25
4.4 Moderation Notifications . . . 25
4.5 Anonymous Access . . . 25

5 Session Management **26**
5.1 Session Expiry Rule . . . 26
5.2 Session Validation . . . 26
5.3 Route-Level Session Helper . . . 26

6 Implementation Notes **27**
6.1 Prisma Setup . . . 27
6.2 Required Validations . . . 28
6.3 Error Responses . . . 28

1 Introduction

This document defines the minimal viable product (MVP) specification for StruggleForum, including the core data models, API endpoints, permission rules, and session management requirements. The specification includes technology-specific implementation details using the chosen stack.

1.1 Technology stack

To implement the MVP for the StruggleForum the following technologies are used:

1. Postgres 16 for the DB
2. NextJS framework for backend and frontend
3. Prisma ORM to have easily written queries to the db using TS(JS).
4. ReactJS library for the frontend
5. TypeScript for more strict typing

1.2 Backend structure

- `app/api/*` contains thin Next.js route handlers only
- `src/features/auth/*` contains authentication and session lifecycle logic
- `src/features/users/*` contains user resource and profile logic
- `src/features/posts/*`, `src/features/comments/*`, and `src/features/categories/*` contain domain business logic
- `src/features/admin/*` contains privileged moderation and administrative actions
- `src/server/*` contains shared infrastructure such as Prisma, auth guards, and HTTP helpers

The API surface is split into four groups:

- `/api/auth/*`: identity and session lifecycle
- `/api/account/*`: actions on the current authenticated user's own account
- `/api/users/*`: user resources and related user data
- `/api/admin/*`: admin-only operations

2 Data Models

This section describes the core entities in the forum system.

2.1 User

- **Fields:**
  - `id`: String (UUID, default: uuid())
  - `username`: String (unique)
  - `email`: String (unique)
  - `avatar_url`: String (optional)
  - `password_hash`: String
  - `role`: Enum (USER, ADMIN) - default: USER
  - `created_at`: DateTime (default: now())
  - `updated_at`: DateTime (updatedAt)
- **Constraints:**
  - Username must be unique
  - Email must be unique and valid

Prisma Schema

```
model User {
  id            String    @id @default(uuid())
  username      String    @unique
  email         String    @unique
  avatarUrl     String?   @map("avatar_url")
  passwordHash  String    @map("password_hash")
  role          Role      @default(USER)
  createdAt     DateTime  @default(now()) @map("created_at")
  updatedAt     DateTime  @updatedAt @map("updated_at")

  sessions           Session[]
  posts              Post[]
  comments           Comment[]
  likes              Like[]
  notifications      Notification[] @relation("UserNotifications")
  actedNotifications Notification[] @relation("NotificationActor")

  @@map("users")
}

enum Role {
  USER
  ADMIN
}
```

TypeScript Types

```
type User = {
  id: string;
  username: string;
  email: string;
```

```
  avatarUrl: string | null;
  passwordHash: string;
  role: 'USER' | 'ADMIN';
  createdAt: Date;
  updatedAt: Date;
};

type UserAccount = Omit<User, 'passwordHash'>;
type UserAdmin = UserAccount;
type UserPublic = Omit<UserAccount, 'email'>;
```

2.2 Session

- **Fields:**
  - `id`: String (UUID, default: uuid())
  - `user_id`: String (Foreign key to User)
  - `token`: String (unique)
  - `created_at`: DateTime (default: now())
  - `last_activity`: DateTime
  - `expires_at`: DateTime
- **Rules:**
  - Sessions expire after **2 hours of inactivity**
  - Last activity timestamp updates on each authenticated request

Prisma Schema

```
model Session {
  id           String   @id @default(uuid())
  userId       String   @map("user_id")
  token        String   @unique
  createdAt    DateTime @default(now()) @map("created_at")
  lastActivity DateTime @map("last_activity")
  expiresAt    DateTime @map("expires_at")

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@map("sessions")
}
```

TypeScript Type

```
type Session = {
  id: string;
  userId: string;
  token: string;
  createdAt: Date;
```

```
  lastActivity: Date;
  expiresAt: Date;
};
```

2.3 Post

- **Fields:**
  - `id`: String (UUID, default: uuid())
  - `author_id`: String (Foreign key to User)
  - `category_id`: String (Foreign key to Category)
  - `title`: String
  - `content`: String (Text)
  - `locked`: Boolean (default: false)
  - `created_at`: DateTime (default: now())
  - `updated_at`: DateTime (updatedAt)
- **Constraints:**
  - Title is required
  - Content is required
  - Category is required
- **Moderation:**
  - `locked`: Prevents new comments (owner or admin)

Prisma Schema

```
model Post {
  id         String    @id @default(uuid())
  authorId   String    @map("author_id")
  categoryId String    @map("category_id")
  title      String
  content    String    @db.Text
  locked     Boolean   @default(false)
  createdAt  DateTime  @default(now()) @map("created_at")
  updatedAt  DateTime  @updatedAt @map("updated_at")

  author    User     @relation(fields: [authorId], references: [id], onDelete:
    Cascade)
  category  Category @relation(fields: [categoryId], references: [id], onDelete:
    Restrict)
  comments  Comment[]
  likes     Like[]

  @@index([authorId])
  @@index([categoryId])
  @@index([createdAt])
  @@map("posts")
}
```

TypeScript Types

```
type Post = {
  id: string;
  authorId: string;
  categoryId: string;
  title: string;
  content: string;
  locked: boolean;
  createdAt: Date;
  updatedAt: Date;
};

type PostWithRelations = Post & {
  author: UserPublic;
  category: Category;
  _count: {
    likes: number;
    comments: number;
  };
};
```

2.4 Comment

- **Fields:**
  - `id`: String (UUID, default: uuid())
  - `post_id`: String (Foreign key to Post)
  - `author_id`: String (Foreign key to User)
  - `parent_id`: String (Foreign key to Comment, optional)
  - `content`: String (Text)
  - `depth`: Int (default: 0)
  - `locked`: Boolean (default: false)
  - `created_at`: DateTime (default: now())
  - `updated_at`: DateTime (updatedAt)
- **Constraints:**
  - Content is required
- **Nested Replies:**
  - Comments can reply to other comments via `parent_id`
  - Top-level comments have `parent_id = null` and `depth = 0`
  - Each reply increments the depth level
- **Moderation:**
  - `locked`: Prevents replies to this comment (owner or admin)

Prisma Schema

```
model Comment {
  id        String    @id @default(uuid())
  postId    String    @map("post_id")
  authorId  String    @map("author_id")
  parentId  String?   @map("parent_id")
  content   String    @db.Text
  depth     Int       @default(0)
  locked    Boolean   @default(false)
  createdAt DateTime  @default(now()) @map("created_at")
  updatedAt DateTime  @updatedAt @map("updated_at")

  post    Post     @relation(fields: [postId], references: [id], onDelete: Cascade)
  author  User     @relation(fields: [authorId], references: [id], onDelete: Cascade
    )
  parent  Comment? @relation("CommentReplies", fields: [parentId], references: [id],
     onDelete: Cascade)
  replies Comment[] @relation("CommentReplies")
  likes   Like[]

  @@index([postId])
  @@index([authorId])
  @@index([parentId])
  @@map("comments")
}
```

TypeScript Types

```
type Comment = {
  id: string;
  postId: string;
  authorId: string;
  parentId: string | null;
  content: string;
  depth: number;
  locked: boolean;
  createdAt: Date;
  updatedAt: Date;
};

type CommentWithAuthor = Comment & {
  author: UserPublic;
  _count: {
    likes: number;
  };
};

type CommentWithReplies = CommentWithAuthor & {
  replies: CommentWithAuthor[];
};
```

2.5 Category

- **Fields:**
  - `id`: String (UUID, default: uuid())
  - `name`: String (unique)
  - `description`: String (optional)
  - `created_at`: DateTime (default: now())
- **Relationships:**
  - One-to-many relationship with Post
  - Each post belongs to exactly one category
- **Constraints:**
  - Name must be unique

Prisma Schema

```
model Category {
  id          String   @id @default(uuid())
  name        String   @unique
  description String?
  createdAt   DateTime @default(now()) @map("created_at")

  posts Post[]

  @@map("categories")
}
```

TypeScript Type

```
type Category = {
  id: string;
  name: string;
  description: string | null;
  createdAt: Date;
};

type CategoryWithCount = Category & {
  _count: {
    posts: number;
  };
};
```

2.6 Notification

- **Fields:**
  - `id`: String (UUID, default: uuid())

  - `user_id`: String (Foreign key to User)
  - `actor_id`: String (Foreign key to User, optional)
  - `type`: Enum (COMMENT, LIKE, REPLY, MENTION, MODERATION)
  - `content`: String
  - `post_id`: String (Foreign key to Post, optional)
  - `comment_id`: String (Foreign key to Comment, optional)
  - `is_read`: Boolean (default: false)
  - `created_at`: DateTime (default: now())

- **Rules:**
  - Notifications can reference the acting user through `actor_id`
  - Notifications may reference a related post or comment through explicit foreign keys
  - The schema must not rely on a generic `reference_id` field for polymorphic references
  - Moderation notifications may omit `post_id` and `comment_id` so the notification remains visible after the moderated content is deleted

Prisma Schema

```
model Notification {
  id          String           @id @default(uuid())
  userId      String           @map("user_id")
  actorId     String?          @map("actor_id")
  type        NotificationType
  content     String
  postId      String?          @map("post_id")
  commentId   String?          @map("comment_id")
  isRead      Boolean          @default(false) @map("is_read")
  createdAt   DateTime         @default(now()) @map("created_at")

  user    User     @relation("UserNotifications", fields: [userId], references: [id
    ], onDelete: Cascade)
  actor   User?    @relation("NotificationActor", fields: [actorId], references: [id
    ], onDelete: SetNull)
  post    Post?    @relation(fields: [postId], references: [id], onDelete: Cascade)
  comment Comment? @relation(fields: [commentId], references: [id], onDelete:
    Cascade)

  @@index([userId, isRead])
  @@index([actorId])
  @@index([postId])
  @@index([commentId])
  @@map("notifications")
}

enum NotificationType {
  COMMENT
  LIKE
  REPLY
  MENTION
  MODERATION
}
```

TypeScript Type

```
type Notification = {
  id: string;
  userId: string;
  actorId: string | null;
  type: 'COMMENT' | 'LIKE' | 'REPLY' | 'MENTION' | 'MODERATION';
  content: string;
  postId: string | null;
  commentId: string | null;
  isRead: boolean;
  createdAt: Date;
};
```

2.7 Like

- **Fields:**
  - `id`: String (UUID, default: uuid())
  - `user_id`: String (Foreign key to User)
  - `post_id`: String (Foreign key to Post, optional)
  - `comment_id`: String (Foreign key to Comment, optional)
  - `created_at`: DateTime (default: now())
- **Constraints:**
  - User can like an entity only once
  - Either postId or commentId must be set (not both)

Prisma Schema

```
model Like {
  id        String   @id @default(uuid())
  userId    String   @map("user_id")
  postId    String?  @map("post_id")
  commentId String?  @map("comment_id")
  createdAt DateTime @default(now()) @map("created_at")

  user    User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  post    Post?    @relation(fields: [postId], references: [id], onDelete: Cascade)
  comment Comment? @relation(fields: [commentId], references: [id], onDelete:
    Cascade)

  @@unique([userId, postId])
  @@unique([userId, commentId])
  @@index([userId])
  @@index([postId])
  @@index([commentId])
  @@map("likes")
}
```

TypeScript Type

```
type Like = {
  id: string;
  userId: string;
  postId: string | null;
  commentId: string | null;
  createdAt: Date;
};
```

3 API Endpoints

This section defines the required endpoints for the forum API.

3.1 Authentication Endpoints

3.1.1 POST /api/auth/register

File: `app/api/auth/register/route.ts`

Request:

```
{
  "username": "string",
  "email": "string",
  "password": "string",
  "avatarUrl": "string|null"
}
```

Response (201):

```
{
  "user": {
    "id": "uuid",
    "username": "string",
    "email": "string",
    "avatarUrl": "string|null",
    "role": "USER",
    "createdAt": "2026-02-02T10:00:00.000Z",
    "updatedAt": "2026-02-02T10:00:00.000Z"
  },
  "token": "string"
}
```

Route Example

```
import { NextRequest, NextResponse } from 'next/server';
import { prisma } from '@/src/server/db/prisma';
import { parseJson } from '@/src/server/http/validation';
import { toErrorResponse } from '@/src/server/http/errors';
import { register } from '@/src/features/auth/service';
```

```
import { RegisterBodySchema } from '@/src/features/auth/validation';

export async function POST(req: NextRequest) {
  const body = await parseJson(req, RegisterBodySchema);

  if (!body.ok) {
    return body.res;
  }

  try {
    const result = await register(prisma, body.data);
    return NextResponse.json(result, { status: 201 });
  } catch (error) {
    return toErrorResponse(error, 'Failed to register user.');
  }
}
```

3.1.2 POST /api/auth/login

Request:

```
{
  "email": "string",
  "password": "string"
}
```

Response (200):

```
{
  "user": {
    "id": "uuid",
    "username": "string",
    "email": "string",
    "avatarUrl": "string|null",
    "role": "USER",
    "createdAt": "timestamp",
    "updatedAt": "timestamp"
  },
  "token": "string"
}
```

3.1.3 POST /api/auth/logout

Headers: Authorization: Bearer {token}

Response (200):

```
{
  "message": "Logged out successfully"
}
```

3.1.4 GET /api/auth/me

Headers: Authorization: Bearer {token}
Purpose: Returns the authenticated user attached to the current session.
Response (200):

```
{
  "id": "uuid",
  "username": "string",
  "email": "string",
  "avatarUrl": "string|null",
  "role": "USER"
}
```

3.2 Account Endpoints

3.2.1 GET /api/account/profile

Headers: Authorization: Bearer {token}
Purpose: Returns the currently authenticated user's own profile.
Response (200):

```
{
  "id": "uuid",
  "username": "string",
  "email": "string",
  "avatarUrl": "string|null",
  "role": "USER",
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

3.2.2 PATCH /api/account/profile

Headers: Authorization: Bearer {token}
Request:

```
{
  "username": "string",
  "email": "string",
  "avatarUrl": "string|null"
}
```

Response (200):

```
{
  "id": "uuid",
  "username": "string",
  "email": "string",
  "avatarUrl": "string|null",
  "role": "USER",
  "updatedAt": "timestamp"
}
```

3.2.3 PATCH /api/account/password

Headers: Authorization: Bearer {token}

Request:

```
{
  "currentPassword": "string",
  "newPassword": "string"
}
```

Response (200):

```
{
  "message": "Password updated successfully"
}
```

3.3 User Endpoints

3.3.1 GET /api/users/:id

Purpose: Returns a public user resource by id. Public user resources do not expose email addresses.

Response (200):

```
{
  "id": "uuid",
  "username": "string",
  "avatarUrl": "string|null",
  "role": "USER",
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

3.3.2 GET /api/users/:id?include=posts

Headers: Authorization: Bearer {token}

Purpose: Returns a user resource with related data. Allowed values for `include` are `posts`, `comments`, and `sessions`. The `sessions` expansion is private and available only to the user themselves or an administrator.

Response (200):

```
{
  "id": "uuid",
  "username": "string",
  "avatarUrl": "string|null",
  "role": "USER",
  "posts": []
}
```

3.4 Post Endpoints

3.4.1 GET /api/posts

Query Parameters:

- `page`: Page number (default: 1)
- `limit`: Items per page (default: 20)
- `category`: Filter by category ID

Response (200):

```
{
  "posts": [
    {
      "id": "uuid",
      "title": "string",
      "content": "string",
      "locked": false,
      "createdAt": "timestamp",
      "updatedAt": "timestamp",
      "author": { "id": "uuid", "username": "string" },
      "category": { "id": "uuid", "name": "string" },
      "_count": { "likes": 0, "comments": 0 }
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 100
  }
}
```

3.4.2 GET /api/posts/:id

Response (200):

```
{
  "id": "uuid",
  "title": "string",
  "content": "string",
  "locked": false,
  "createdAt": "timestamp",
  "updatedAt": "timestamp",
  "author": { "id": "uuid", "username": "string" },
  "category": { "id": "uuid", "name": "string" },
  "_count": { "likes": 0, "comments": 0 }
}
```

3.4.3 POST /api/posts

Headers: Authorization: Bearer {token}

Request:

```
{
  "title": "string",
  "content": "string",
  "categoryId": "uuid"
}
```

Response (201):

```
{
  "id": "uuid",
  "authorId": "uuid",
  "categoryId": "uuid",
  "title": "string",
  "content": "string",
  "locked": false,
  "createdAt": "timestamp",
  "updatedAt": "timestamp",
  "author": { "id": "uuid", "username": "string" },
  "category": { "id": "uuid", "name": "string" },
  "_count": { "likes": 0, "comments": 0 }
}
```

3.4.4 PUT /api/posts/:id

Headers: Authorization: Bearer {token}

Request:

```
{
  "title": "string",
  "content": "string",
  "categoryId": "uuid"
}
```

Response (200):

```
{
  "id": "uuid",
  "authorId": "uuid",
  "categoryId": "uuid",
  "title": "string",
  "content": "string",
  "locked": false,
  "createdAt": "timestamp",
  "updatedAt": "timestamp",
  "category": { "id": "uuid", "name": "string" },
  "author": { "id": "uuid", "username": "string" },
  "_count": { "likes": 0, "comments": 0 }
}
```

3.4.5 DELETE /api/posts/:id

Headers: Authorization: Bearer {token} (Owner only)
Response (200):

```
{
  "message": "Post deleted successfully"
}
```

3.4.6 POST /api/posts/:id/lock

Headers: Authorization: Bearer {token} (Owner or admin)
Response (200):

```
{
  "id": "uuid",
  "locked": true
}
```

3.4.7 POST /api/posts/:id/unlock

Headers: Authorization: Bearer {token} (Owner or admin)
Response (200):

```
{
  "id": "uuid",
  "locked": false
}
```

3.5 Comment Endpoints

3.5.1 GET /api/posts/:id/comments

Response (200):

```
{
  "comments": [
    {
      "id": "uuid",
      "content": "string",
      "author": { "id": "uuid", "username": "string" },
      "parentId": null,
      "depth": 0,
      "locked": false,
      "createdAt": "timestamp",
      "updatedAt": "timestamp",
      "_count": { "likes": 0 },
      "replies": [
        {
          "id": "uuid",
          "content": "string",
          "author": { "id": "uuid", "username": "string" },
```

```
          "parentId": "parent-uuid",
          "depth": 1,
          "locked": false,
          "createdAt": "timestamp",
          "updatedAt": "timestamp",
          "_count": { "likes": 0 },
          "replies": []
        }
      ]
    }
  ]
}
```

3.5.2 POST /api/posts/:id/comments

Headers: Authorization: Bearer {token}

Request:

```
{
  "content": "string",
  "parentId": "uuid" // Optional: for nested replies
}
```

Response (201):

```
{
  "id": "uuid",
  "postId": "uuid",
  "authorId": "uuid",
  "content": "string",
  "author": { "id": "uuid", "username": "string" },
  "parentId": "uuid", // null for top-level comments
  "depth": 0,
  "locked": false,
  "createdAt": "timestamp",
  "updatedAt": "timestamp",
  "_count": { "likes": 0 }
}
```

3.5.3 PUT /api/comments/:id

Headers: Authorization: Bearer {token}

Request:

```
{
  "content": "string"
}
```

Response (200):

```
{
  "id": "uuid",
```

```
  "content": "string",
  "updatedAt": "timestamp",
  "_count": { "likes": 0 }
}
```

3.5.4 DELETE /api/comments/:id

Headers: Authorization: Bearer {token} (Owner only)
Response (200):

```
{
  "message": "Comment deleted successfully"
}
```

3.5.5 POST /api/comments/:id/lock

Headers: Authorization: Bearer {token} (Owner or admin)
Response (200):

```
{
  "id": "uuid",
  "locked": true
}
```

3.5.6 POST /api/comments/:id/unlock

Headers: Authorization: Bearer {token} (Owner or admin)
Response (200):

```
{
  "id": "uuid",
  "locked": false
}
```

3.6 Like Endpoints

3.6.1 POST /api/likes

Headers: Authorization: Bearer {token}
Request:

```
{
  "targetType": "post|comment",
  "targetId": "uuid"
}
```

Response (201):

```
{
  "id": "uuid",
  "targetType": "post|comment",
  "targetId": "uuid",
  "createdAt": "timestamp"
}
```

3.6.2 DELETE /api/likes/:id

Headers: Authorization: Bearer {token}
Response (200):

```
{
  "message": "Like removed successfully"
}
```

3.7 Notification Endpoints

3.7.1 GET /api/notifications

Headers: Authorization: Bearer {token}
Response (200):

```
{
  "notifications": [
    {
      "id": "uuid",
      "type": "COMMENT|LIKE|REPLY|MENTION|MODERATION",
      "content": "string",
      "actorId": "uuid",
      "postId": "uuid|null",
      "commentId": null,
      "isRead": false,
      "createdAt": "timestamp",
      "actor": {
        "id": "uuid",
        "username": "string"
      }
    }
  ]
}
```

3.7.2 PUT /api/notifications/:id/read

Headers: Authorization: Bearer {token}
Response (200):

```
{
  "id": "uuid",
  "isRead": true
}
```

3.8 Category Endpoints

3.8.1 GET /api/categories

Response (200):

```
{
  "categories": [
    {
      "id": "uuid",
      "name": "string",
      "description": "string",
      "createdAt": "timestamp",
      "_count": { "posts": 0 }
    }
  ]
}
```

3.8.2 POST /api/categories

Headers: Authorization: Bearer {token} (Admin only)

Request:

```
{
  "name": "string",
  "description": "string"
}
```

Response (201):

```
{
  "id": "uuid",
  "name": "string",
  "description": "string",
  "createdAt": "timestamp"
}
```

3.9 Admin Endpoints

3.9.1 GET /api/admin/users

Headers: Authorization: Bearer {token} (Admin only)

Response (200):

```
[
  {
    "id": "uuid",
    "username": "string",
    "email": "string",
```

```
    "avatarUrl": "string|null",
    "role": "USER"
  }
]
```

3.9.2 PATCH /api/admin/users/:id/role

Headers: Authorization: Bearer {token} (Admin only)

Request:

```
{
  "role": "USER|ADMIN"
}
```

Response (200):

```
{
  "id": "uuid",
  "role": "ADMIN"
}
```

3.9.3 DELETE /api/admin/posts/:id

Headers: Authorization: Bearer {token} (Admin only)

Request:

```
{
  "reason": "Removed for prohibited advertising"
}
```

Response (200):

```
{
  "message": "Post deleted successfully"
}
```

Behavior:

- Deletes any post regardless of author
- Sends a moderation notification to the post author with the provided reason

3.9.4 DELETE /api/admin/comments/:id

Headers: Authorization: Bearer {token} (Admin only)

Request:

```
{
  "reason": "Removed for prohibited advertising"
}
```

Response (200):

```
{
  "message": "Comment deleted successfully"
}
```

Behavior:

- Deletes any comment regardless of author
- Sends a moderation notification to the comment author with the provided reason

4 Permission Rules

This section defines the authorization rules for different user roles.

4.1 Responsibility Split

- `auth` endpoints handle registration, login, logout, and current-session identity
- `account` endpoints handle only the currently authenticated user's own profile and password
- `users` endpoints expose user resources and related data; they do not create or manage sessions
- `admin` endpoints handle privileged actions such as role changes and moderation

4.2 User Permissions

Regular users (`role: "USER"`) can:

- View all posts and comments (read-only access)
- Create new posts
- Edit their own posts
- Delete their own posts
- Lock and unlock their own posts
- Create comments on any post
- Edit their own comments
- Delete their own comments
- Lock and unlock their own comments
- Like/unlike posts and comments
- View and manage their own notifications
- View their own account profile via `/api/account/profile`
- Update their own profile and password via `/api/account/*`

Users **cannot**:

- Edit or delete posts created by other users
- Edit or delete comments created by other users
- Access or manage other users' notifications
- Perform administrative actions
- Change roles
- Access `/api/admin/*` endpoints
- Use user-resource endpoints as a replacement for authentication or session endpoints

4.3 Admin Permissions

Administrators (`role: "ADMIN"`) have all user permissions plus:

- Edit any post (regardless of author)
- Delete any post through `/api/admin/posts/:id` with a moderation reason
- Edit any comment
- Delete any comment through `/api/admin/comments/:id` with a moderation reason
- Create categories
- View all users
- Change user roles
- Lock and unlock any post
- Lock and unlock any comment

4.4 Moderation Notifications

- When an admin deletes a post or comment, the author must receive a `MODERATION` notification
- A moderation notification must include the human-readable deletion reason in `content`
- Moderation deletion must not reuse the owner delete endpoints, because owner delete does not require a reason
- The moderation notification should remain available even after the post or comment has been deleted

4.5 Anonymous Access

Unauthenticated users can:

- View all posts (read-only)
- View all comments (read-only)
- View categories

Anonymous users **cannot**:

- Create, edit, or delete any content
- Like posts or comments
- Access notifications
- Access `/api/account/*`
- Access `/api/admin/*`

5 Session Management

5.1 Session Expiry Rule

- Sessions expire after **2 hours of inactivity**
- The `last_activity` timestamp is updated on every authenticated API request
- Inactivity is defined as the time elapsed since the `last_activity` timestamp
- When a session expires:
  - The session token becomes invalid
  - Subsequent requests with the expired token return 401 Unauthorized
  - The user must log in again to obtain a new session token

5.2 Session Validation

On every authenticated request, the system must:

1. Verify the session token exists and is valid
2. Check if the session has expired by comparing `expires_at` to the current time
3. If valid: update both `last_activity` and `expires_at`
4. If expired: return 401 Unauthorized and invalidate the session

5.3 Route-Level Session Helper

File: `src/server/auth/session.ts`

```
export const SESSION_TTL_MS = 2 * 60 * 60 * 1000;

export function extractBearerToken(req: Pick<NextRequest, 'headers'>) {
  const authorization = req.headers.get('authorization');

  if (!authorization) {
    return null;
  }

  const [scheme, token] = authorization.split(' ');

  if (scheme?.toLowerCase() !== 'bearer' || !token) {
    return null;
  }

  return token;
```

```
}

export async function requireSession(prisma: PrismaClient, req: NextRequest) {
  const token = extractBearerToken(req);

  if (!token) {
    throw new UnauthorizedError();
  }

  const session = await getSessionByToken(prisma, token);

  if (!session) {
    throw new UnauthorizedError('Invalid session');
  }

  const now = new Date();

  if (session.expiresAt <= now) {
    await deleteSessionById(prisma, session.id);
    throw new UnauthorizedError('Session expired');
  }

  return updateSessionActivity(
    prisma,
    session.id,
    now,
    new Date(now.getTime() + SESSION_TTL_MS),
  );
}
```

6 Implementation Notes

6.1 Prisma Setup

File: `src/server/db/prisma.ts`

```
import 'server-only';
import { PrismaClient } from '@prisma/client';
import { PrismaPg } from '@prisma/adapter-pg';
import { Pool } from 'pg';

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

const pool = new Pool({ connectionString: process.env.DATABASE_URL });
const adapter = new PrismaPg(pool);

export const prisma = new PrismaClient({
  adapter,
  log: ['query', 'error', 'warn'],
});

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

Environment Variables (.env):

```
DATABASE_URL="postgresql://user:password@localhost:5432/struggleforum?schema=public"
CORS_ALLOWED_ORIGINS="http://localhost:3000,http://localhost:3001"
```

Required npm packages:

```
npm install @prisma/client
npm install @prisma/adapter-pg pg bcryptjs zod
npm install -D prisma
npm install -D @types/bcryptjs
npm install dotenv
```

Prisma Commands:

```
# Initialize Prisma
npx prisma init

# Create migration
npx prisma migrate dev --name init

# Generate Prisma Client
npx prisma generate

# Open Prisma Studio
npx prisma studio
```

6.2 Required Validations

- Email format validation on registration
- Password strength requirements (minimum 8 characters)
- UUID validation for route parameters and relation identifiers
- Role and ownership checks on protected writes
- Rate limiting on authentication endpoints

6.3 Error Responses

Most error responses follow this format:

```
{
  "error": "Human-readable error message"
}
```

Validation failures may additionally return an `issues` payload from the request parser. Common HTTP status codes:

- 400: Bad Request (validation errors)
- 401: Unauthorized (authentication required/failed)

- 403: Forbidden (insufficient permissions)
- 404: Not Found
- 409: Conflict (duplicate or already existing resource)
- 503: Service Unavailable (database unavailable)
- 500: Internal Server Error